Student Name	Srikanth V
Register Number	TU-6253210211054
Project Title	Multi-Echelon Supply Chain Network Design via MILP
Week	Week 3

WEEK-3 TASK

Task Name

Multi-Echelon Supply Chain Network Design via Mixed-Integer Linear Programming (MILP)

Abstract

A Multi-Echelon Supply Chain Network Design model uses Mixed-Integer Linear Programming (MILP) to determine an efficient flow of products from factories through warehouses to customer zones. The model considers transportation costs, factory production capacities, warehouse storage capacities, route limits, customer demand, and minimum service-level requirements. The objective is to minimize total supply chain operating expenditure while satisfying business constraints.

Introduction

Supply chain network design is an important Business Analytics problem because companies must make cost-effective decisions about production, warehousing, transportation, and customer service. MILP combines continuous transportation decisions with binary warehouse-opening decisions. By analyzing supply chain data mathematically, the model helps management select an efficient network and allocate products while maintaining required service levels.

Objectives

1. To minimize total supply chain operating expenditure.
2. To determine the optimal flow of products from factories to warehouses.
3. To determine the optimal distribution from warehouses to customer zones.
4. To satisfy customer demand and minimum service-level requirements.
5. To respect factory, warehouse, and transportation capacity constraints.
6. To support data-driven supply chain decision-making.

MILP Model

Decision Variables

x_{fw} = quantity transported from factory f to warehouse w.

y_{wc} = quantity transported from warehouse w to customer c.

z_w = 1 if warehouse w is opened, otherwise 0.

Objective Function

```
Minimize Z =
Σ(f,w) C_fw x_fw
+ Σ(w,c) C_wc y_wc
+ Σ(w) FC_w z_w
```

The objective minimizes factory-to-warehouse transportation cost, warehouse-to-customer transportation cost, and fixed warehouse operating/opening cost.

Constraints

1. Factory Capacity

```
Σ(w) x_fw ≤ S_f     for every factory f
```

2. Warehouse Flow Balance

```
Σ(f) x_fw = Σ(c) y_wc     for every warehouse w
```

3. Warehouse Capacity

```
Σ(f) x_fw ≤ K_w z_w     for every warehouse w
```

4. Customer Demand

```
Σ(w) y_wc ≥ D_c     for every customer c
```

5. Factory-to-Warehouse Route Limit

```
x_fw ≤ U_fw     for every factory f and warehouse w
```

6. Warehouse-to-Customer Route Limit

```
y_wc ≤ U_wc     for every warehouse w and customer c
```

7. Minimum Service Level

```
Σ(w) y_wc ≥ SL_c × D_c     for every customer c
```

8. Variable Domains

```
x_fw ≥ 0, y_wc ≥ 0, and z_w ∈ {0,1}
```

Constraint Matrix Representation

The MILP can be represented in standard optimization form:

```
Minimize:     c■X

Subject to:   AX ≤ b
              X ≥ 0
              z ∈ {0,1}
```

Constraint	Business Meaning
Factory capacity	Limits production and outbound shipments

Constraint	Business Meaning
Warehouse flow balance	Ensures inbound flow equals outbound flow
Warehouse capacity	Limits warehouse handling/storage
Customer demand	Ensures required customer demand is served
Route limits	Limits quantity on individual transport routes
Service level	Maintains minimum percentage of demand served
Binary decision	Determines whether a warehouse is opened

Sample Business Data

Factories

Factory	Capacity
F1	500 units
F2	600 units

Warehouses

Warehouse	Capacity	Fixed Cost
W1	700 units	■ 10,000
W2	600 units	■ 8,000

Customer Zones

Customer	Demand	Minimum Service Level
C1	400 units	95%
C2	300 units	90%
C3	300 units	95%

Service-Level Calculations

C1 requires at least 400 × 0.95 = 380 units, C2 requires at least 300 × 0.90 = 270 units, and C3 requires at least 300 × 0.95 = 285 units.

Python Code

```
import pulp

model = pulp.LpProblem("Multi_Echelon_Supply_Chain", pulp.LpMinimize)

factories = ["F1", "F2"]
warehouses = ["W1", "W2"]
customers = ["C1", "C2", "C3"]

capacity = {"F1": 500, "F2": 600}
demand = {"C1": 400, "C2": 300, "C3": 300}
warehouse_capacity = {"W1": 700, "W2": 600}
fixed_cost = {"W1": 10000, "W2": 8000}
service = {"C1": 0.95, "C2": 0.90, "C3": 0.95}

cost_fw = {
    ("F1","W1"): 10, ("F1","W2"): 15,
    ("F2","W1"): 12, ("F2","W2"): 8
}

cost_wc = {
    ("W1","C1"): 6, ("W1","C2"): 7, ("W1","C3"): 9,
    ("W2","C1"): 8, ("W2","C2"): 5, ("W2","C3"): 6
}

x = pulp.LpVariable.dicts("Factory_Warehouse", cost_fw.keys(), lowBound=0)
y = pulp.LpVariable.dicts("Warehouse_Customer", cost_wc.keys(), lowBound=0)
z = pulp.LpVariable.dicts("Warehouse_Open", warehouses, cat="Binary")

model += (
    pulp.lpSum(cost_fw[f,w] * x[f,w] for f,w in cost_fw)
    + pulp.lpSum(cost_wc[w,c] * y[w,c] for w,c in cost_wc)
    + pulp.lpSum(fixed_cost[w] * z[w] for w in warehouses)
```

```
)

for f in factories:
    model += pulp.lpSum(x[f,w] for w in warehouses) <= capacity[f]

for w in warehouses:
    model += pulp.lpSum(x[f,w] for f in factories) == pulp.lpSum(y[w,c] for c in customers)
    model += pulp.lpSum(x[f,w] for f in factories) <= warehouse_capacity[w] * z[w]

for c in customers:
    model += pulp.lpSum(y[w,c] for w in warehouses) >= service[c] * demand[c]

model.solve()

print("Status:", pulp.LpStatus[model.status])
print("Minimum Cost:", pulp.value(model.objective))
```

Simulation Results & Observations

Cost Optimization

The MILP model identifies a feasible network configuration that minimizes total operating expenditure based on the supplied cost and capacity data.

Network Decision

The binary variables determine which warehouses should be opened, while the continuous variables determine product quantities on each route.

Service Level

The solution must satisfy the minimum service-level requirement for every customer zone.

Capacity Utilization

Factory and warehouse capacity constraints prevent the model from assigning more products than available operational capacity.

Benefits

• Reduced transportation and warehouse operating costs.

• Better utilization of factories and warehouses.

• Improved customer service planning.

• Data-driven logistics network decisions.

• Improved visibility of supply chain trade-offs.

Conclusion

The Multi-Echelon Supply Chain Network Design model demonstrates how Business Analytics and Mixed-Integer Linear Programming can support strategic logistics decisions. By combining cost, capacity, demand, route limits, warehouse decisions, and service-level requirements, the model helps identify an efficient supply chain network. This provides management with a structured, data-driven approach to reducing operating expenditure while maintaining customer service.